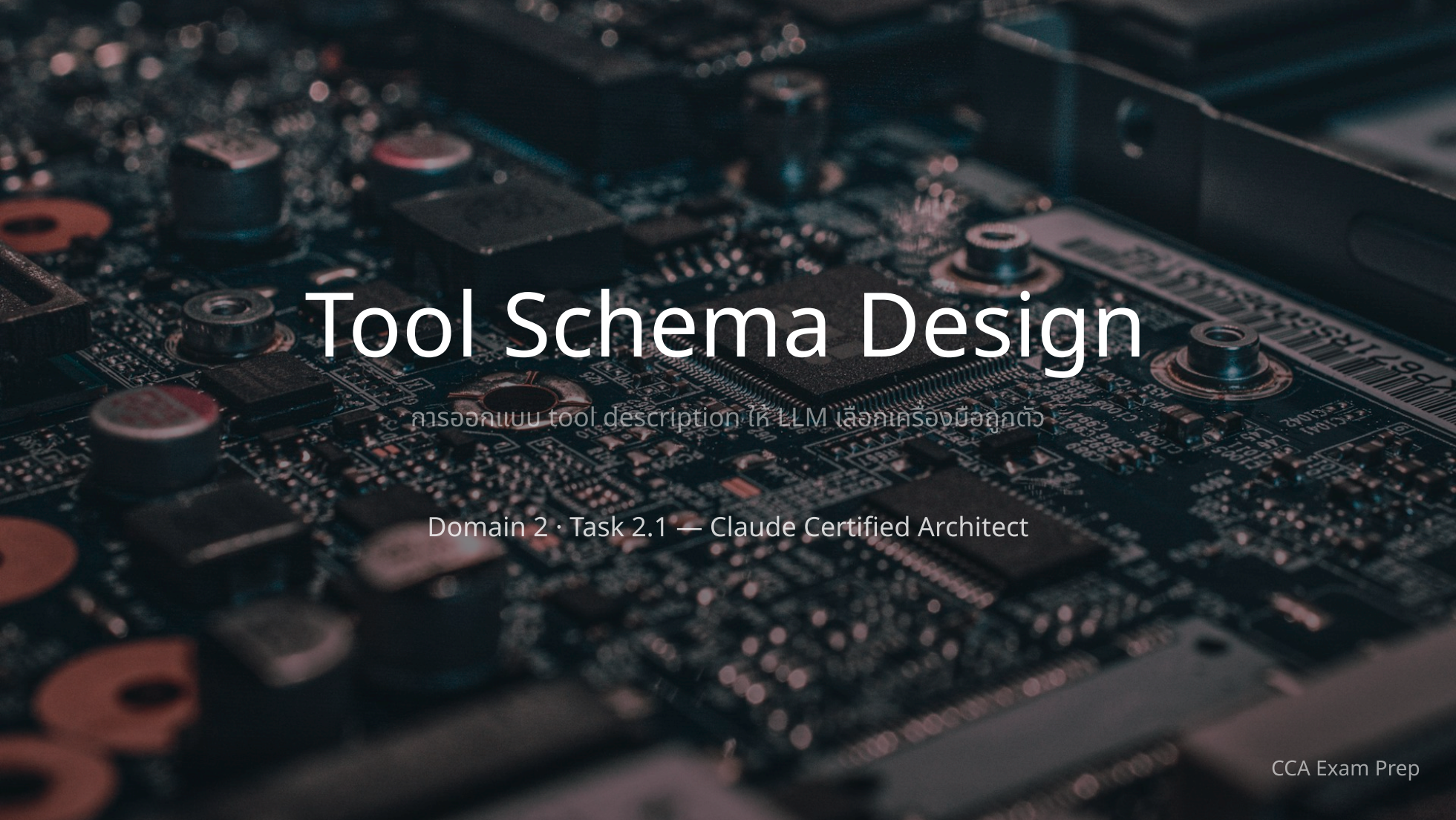




Tool Schema Design

การออกแบบ tool description ให้ LLM เลือกเครื่องมือถูกตัว

Domain 2 · Task 2.1 — Claude Certified Architect

Tool Description คืออะไร และทำไมสำคัญ

Key Concept

tool description คือกลไก **PRIMARY** ที่ LLM ใช้ในการเลือกเครื่องมือ (tool selection)

ไม่ใช่ metadata เสริมที่มีก็ได้ไม่มีก็ได้

"Tool descriptions are the PRIMARY mechanism LLMs use for tool selection. This is not supplementary metadata."

โมเดลตัดสินใจเรียก tool ไหน**จากคำอธิบายเป็นหลัก** → คำอธิบาย**ถ้าถาม** = โมเดล**เลือกผิด**

5 องค์ประกอบของ Description ระดับ Production

description ที่เชื่อถือได้ต้องมีครบทั้ง 5 องค์ประกอบ

- 1 **Primary purpose** — บอกจุดประสงค์หลักอย่างชัดเจน ไม่กำกวม
- 2 **Input specifications** — ชนิดข้อมูล, รูปแบบ (format), ข้อจำกัด, field ไหน required/optional
- 3 **Example queries** — ตัวอย่างการใช้งานจริงเพื่อยึด (anchor) ความเข้าใจของโมเดล
- 4 **Edge cases & limitations** — บอกว่า tool นี้ "ไม่" ทำอะไร
- 5 **Explicit boundaries** — เมื่อไหร่ใช้ tool นี้ vs. tool อื่นที่คล้ายกัน

Minimal vs. Production-Grade

✗ Minimal (มีปัญห)

`get_customer`

"Retrieves customer information"

`lookup_order`

"Retrieves order details"

สั้น · คำควม · ไม่มี format · ไม่มีขอบเขต

✓ Production-grade

`get_customer`

"Looks up a customer account by email, phone, or customer ID. Returns customer profile... Use this when you need to verify who the customer is. **Do NOT use for order-specific queries — use lookup_order for those.**"

👉 ประโยค **Do NOT use...** คือ **explicit boundary** ที่กั้นการเลือกผิด



The Misrouting Problem

อาการ

agent เรียก tool **ผิดตัว** — query "check my order #12345" กลับเข้า `get_customer` แทนที่จะเป็น `lookup_order`

สาเหตุ

description ของสอง tool **ทับซ้อนกัน (overlapping)** และทั้งคู่รับ identifier ที่คล้ายกัน

→ ไม่มีเส้นแบ่งชัดเจนว่าอันไหนใช้เมื่อไหร่ โมเดลจึง**สับสน**

✅ วิธีแก้

ขยาย description ให้ครบ 5 องค์ประกอบ โดยเฉพาะ **explicit boundary**

งานน้อย แต่ได้ผลสูง — **low effort, high leverage**

⚠ วิธีแก้ที่ถูกต้อง vs. วิธีที่ผิด

★ ออกสอบบ่อย

✅ คำตอบที่ถูกต้องเสมอเมื่อเจอ **misrouting**
แก้ **description** ก่อนเป็นอันดับแรก

× **Few-shot examples**

เพิ่ม token overhead แต่**ไม่แก้ต้นเหตุ**ความกำกวม

× **Routing classifier**

over-engineer เกินจำเป็น และไป **bypass** ความเข้าใจของ LLM

⚡ **Tool consolidation (ยุบรวม tool)**

ใช้ effort มากกว่าการแก้ description เอะ — เป็นทางเลือกระยะยาวได้ แต่**ไม่ใช่ first fix**

Tool Splitting Pattern

เมื่อ tool หนึ่งตัวทำงาน**กว้างเกินไป** → แยกเป็นหลาย tool
ที่แต่ละตัวมีจุดประสงค์**แคบและชัด**

Before

`analyze_document`

"Analyses a document and returns results"

กว้างจนโมเดลไม่รู้จะใช้เมื่อไหร่

After → แยกเป็น 3 tool

`extract_data_points`

ดึงข้อมูลมีโครงสร้าง (วันที่ จำนวนเงิน ชื่อ)

`summarize_content`

สรุปประเด็นหลัก

`verify_claim_against_source`

ตรวจว่า claim ถูก support โดยเอกสารต้นทางหรือไม่

แต่ละตัวมีขอบเขตชัดเจน

Tool Renaming Strategy

บางครั้ง**ไม่ต้องเขียน logic ใหม่** — แค่เปลี่ยนชื่อ tool ที่คล้ายกันจนสับสน ก็ขจัด **functional overlap** ได้



ทำให้จุดประสงค์**ชัดเจน** โดยไม่ต้องแก้ implementation เลย

System Prompt Conflicts

จุดที่มักมองข้าม

Key Concept

คำสั่งใน **system prompt** ที่ไต่ต่อ keyword สามารถ **override** description ที่เขียนไว้ดีแล้วได้

ตัวอย่าง

ประโยค *"always check customer details before proceeding"*

→ อาจกระตุ้นให้โมเดลเชื่อมโยงเข้ากับ `get_customer` โดย**ไม่สนใจ** description จริง

Critical practice

หลังอัปเดต description **ทุกครั้ง** → กลับไป review system prompt เพื่อหา keyword ที่ขัดกัน

! Exam Traps ที่ต้องระวัง

1. Few-shot examples trap

เพิ่ม overhead แต่ไม่แก้ความกำกวม — ต้องแก้ **description** ก่อน

2. Routing classifier trap

over-engineer ไปใส่ infrastructure ที่ไม่จำเป็นตั้งแต่แรก

3. Tool consolidation trap

เป็นทางเลือกระยะยาวที่ valid แต่ใช้ effort มากกว่าการขยาย description

4. System prompt oversight

ลืมตรวจ keyword conflict ใน system prompt หลังแก้ description



Practice Scenario

โจทย์: production log แสดงว่า `get_customer` ถูกเรียกผิดสำหรับ query เกี่ยวกับ order ("check my order #12345") ทั้งสอง tool มี description แบบ minimal และรับ identifier ที่คล้ายกัน

first step ที่ได้ผลที่สุดในการเพิ่มความแม่นยำของ tool selection คืออะไร?

- A. เพิ่ม few-shot examples 5-8 ตัวลงใน system prompt
- B. ขยาย description ให้มี input format, examples, edge cases และ boundaries
- C. ยุบสอง tool รวมเป็น `lookup_entity` ตัวเดียว
- D. สร้าง routing layer ที่ parse input ด้วย keyword detection

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปสไลด์ถัดไปเพื่อเฉลย



เฉลย Practice Scenario

คำตอบที่ถูก — Option B

ขยาย description ให้ครบ (input format, examples, edge cases, explicit boundaries)

เพราะ description คือ**กลไกหลัก**ที่โมเดลใช้เลือก tool · ต้นเหตุคือ description กำกวม → ต้องแก้ที่ต้นเหตุ · งานน้อยได้ผลสูง

A — เพิ่ม token overhead แต่ไม่แก้ความกำกวมที่ต้นเหตุ

C — consolidation ใช้ effort มากกว่ามาก ไม่ใช่ first fix

D — routing classifier over-engineer และ bypass ความเข้าใจของ LLM



Build Exercise (30 นาที)

ออกแบบ description ขจัด Misrouting

🎯 Learning objectives

เข้าใจว่า description ขับเคลื่อน tool selection · เขียน description ระดับ production ครบ 5 องค์ประกอบ · วิจัยภัย misrouting · หา conflict ใน system prompt

- 1 สร้าง 2 MCP tool ที่ description กำหนดโดยตั้งใจ
- 2 ทดสอบด้วย 10 query แล้ว log การเลือก tool (จะเห็น misroute 2-3 ครั้ง)
- 3 เขียน description ใหม่ครบ 5 องค์ประกอบ (3-5 ประโยค · ระบุ format · ประโยค "Do NOT use...")
- 4 รัน 10 query เดิมซ้ำ เทียบความแม่นยำ (ควรขึ้นเป็น 9/10 หรือ 10/10)
- 5 review system prompt หา keyword-sensitive instruction แล้วเขียนใหม่



Exam Sim — ข้อ 1/5 (ลองคิดก่อนดูเฉลย)

production log แสดงว่า agent เรียก `get_customer` บ่อยครั้งเมื่อผู้ใช้ถามเรื่อง order (เช่น "check my order #12345") แทนที่จะเรียก `lookup_order` ทั้งสอง tool มี description แบบ minimal ("Retrieves customer information" / "Retrieves order details") และรับ identifier รูปแบบคล้ายกัน · **first step ที่ได้ผลที่สุด**ในการเพิ่มความน่าเชื่อถือของ tool selection คืออะไร?

- A. เพิ่ม few-shot examples 5-8 ตัวลงใน system prompt เพื่อสาธิตรูปแบบการเลือก tool ที่ถูกต้องสำหรับ query เกี่ยวกับ order
- B. ขยาย description ของแต่ละ tool ให้มี input format, example queries, edge cases และ boundaries ว่าเมื่อไหร่ใช้ตัวนี้ vs. tool ที่คล้ายกัน
- C. สร้าง routing layer ที่ parse input ก่อนทุก turn แล้ว pre-select tool จาก keyword ที่ตรวจพบ
- D. ยูนสอง tool รวมเป็น `lookup_entity` ตัวเดียวที่รับ identifier ใดก็ได้ แล้วตัดสินใจภายในว่าจะ query backend ไหน

✓ เฉลย Exam Sim — ข้อ 1/5

คำตอบที่ถูก: Option B

tool description คือกลไกหลักที่ LLM ใช้เลือก tool การขยาย description จึงเป็นการแก้ที่ต้นเหตุของการ misrouting โดยตรง — งานน้อยแต่ได้ผลสูงที่สุด

ทำไมข้ออื่นผิด

- A — few-shot เพิ่ม token overhead แต่ไม่แก้ต้นเหตุ เพราะ description ยังไม่แยกความต่างของสอง tool
- C — routing layer over-engineer เกินไปสำหรับ first step และ bypass ความเข้าใจภาษาธรรมชาติของ LLM
- D — consolidation เป็นทางเลือกที่ valid แต่ใช้ effort มากกว่าการขยาย description มาก ไม่ใช่ first step ที่ได้สัดส่วน



Exam Sim — ข้อ 2/5 (ลองคิดก่อนดูเฉลย)

tool ชื่อ `analyse_document` มี description ว่า "Analyses a document and returns results." ผู้ใช้รายงานว่า agent บางครั้ง extract data ทั้งที่ต้องการ summary และกลับกัน · แนวทางที่ดีที่สุดคืออะไร?

A. เพิ่ม few-shot examples แสดงว่าเมื่อไรควร extract data vs. เมื่อไรควร summarise

B. เปลี่ยนชื่อ tool เป็น `extract_and_summarise_document` เพื่อความชัดเจน

C. แยก tool ออกเป็น tool เฉพาะทาง: `extract_data_points` , `summarise_content` , `verify_claim_against_source` แต่ละตัวมี description ที่แคบ

D. เพิ่ม pre-processing step ที่ตัดสิน user intent ก่อนเรียก tool

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป



เฉลย Exam Sim — ข้อ 2/5

คำตอบที่ถูก: Option C

tool ที่ทำงานกว้างเกินไปสร้างความกำกวม การแตกเป็น tool เฉพาะทางที่มี input/output contract ชัดเจน ให้โมเดลมีตัวเลือกที่แคบและชัดในการเลือก

ทำไมข้ออื่นผิด

- A** — few-shot แก่ที่อาการ ไม่ใช่ต้นเหตุ ต้นเหตุคือ tool เดียวรับผิดชอบหลายหน้าที่
- B** — การเปลี่ยนชื่ออย่างเดียวไม่คลายความกำกวมว่าผู้ใช้งานต้องการ operation ไหน tool ยังทำหลายหน้าที่อยู่ดี
- D** — pre-processing step over-engineer การแก้ควรอยู่ที่ระดับ tool interface ไม่ใช่ routing layer ภายนอก



Exam Sim — ข้อ 3/5 (ลองคิดก่อนดูเฉลย)

หลังปรับปรุง description ของ `get_customer` และ `lookup_order` แล้ว developer พบว่า query เกี่ยวกับ `order` ยังคง route ไปที่ `get_customer` เป็นบางครั้ง · system prompt มีข้อความว่า "Always check customer details before proceeding with any request." · สาเหตุที่น่าจะเป็นคืออะไร?

- A. tool description ต้องมีรายละเอียดมากขึ้นอีกเพื่อ override system prompt
- B. keyword-sensitive instruction ใน system prompt สร้างความเชื่อมโยงกับ tool โดยไม่ตั้งใจ ซึ่ง override description ที่ปรับปรุงแล้ว
- C. โมเดล cache description เก่าไว้ และต้อง context reset
- D. ชื่อ tool คล้ายกันเกินไป ควรเปลี่ยนชื่อทั้งคู่

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 3/5

คำตอบที่ถูก: Option B

keyword-sensitive instruction ใน system prompt สามารถ override description ที่เขียนดีแล้วอย่างเจียบ ๆ วลี "always check customer details" สร้างความเชื่อมโยงระหว่าง query เกี่ยวกับ customer กับ `get_customer`

ทำไมข้ออื่นผิด

- A — ปัญหาไม่ได้อยู่ที่คุณภาพ description ซึ่งปรับปรุงแล้ว แต่อยู่ที่ system prompt ที่ขัดกับมัน
- C — LLM ไม่ cache tool description ระหว่าง API call แต่ละ request ประเมิน description ใหม่ทุกครั้ง
- D — ชื่อทั้งสองแยกกันชัดเจนอยู่แล้ว (`get_customer` vs `lookup_order`) ปัญหาอยู่ที่ system prompt ไม่ใช่ชื่อ



Exam Sim — ข้อ 4/5 (ลองคิดก่อนดูเฉลย)

มีสอง tool ที่ชื่อคล้ายกันจนสับสน: `analyse_content` และ `analyse_web_content` ทั้งคู่จัดการข้อมูล web . วิธีแก้ที่ได้ผลที่สุดคืออะไร?

A. เพิ่ม description ละเอียดให้ทั้งสอง tool อธิบายความแตกต่าง

B. เปลี่ยนชื่อ `analyse_content` เป็น `extract_web_results` พร้อม description เฉพาะ web ทำให้จุดประสงค์ชัดโดยไม่แก้ implementation

C. merge ทั้งสอง tool เป็น `analyse_all_content` ตัวเดียว

D. เพิ่ม routing classifier ที่ตัดสินใจจะใช้ tool ไหนจากเนื้อหา input

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 4/5

คำตอบที่ถูก: Option B

การเปลี่ยนชื่อจัด functional overlap ที่ระดับ interface การตั้งชื่อเป็น `extract_web_results` พร้อม description เฉพาะทำให้จุดประสงค์ชัดเจนโดยไม่ต้องแก้ไข implementation

ทำไมข้ออื่นผิด

- A — เมื่อชื่อคล้ายกันจนสับสน description อย่างเดียวอาจไม่พอ ตัวชื่อเองสร้างความกำกวมที่ description ต้องทำงานหนักเกินไปเพื่อกลบ
- C — การ merge สร้างปัญหา generic-tool ขึ้นใหม่ — tool เดียวรับผิดชอบหลายหน้าที่
- D — routing classifier over-engineer ในเมื่อการเปลี่ยนชื่อง่าย ๆ ก็คลายความกำกวมได้



Exam Sim — ข้อ 5/5 (ลองคิดก่อนดูเฉลย)

tool description ระดับ production ควรมี 5 องค์ประกอบใด?

A. Name, version, author, licence และ dependencies

B. tool ทำอะไร, รับ input อะไร, example queries, edge cases และ explicit boundaries เทียบกับ tool ที่คล้ายกัน

C. Endpoint URL, authentication method, rate limits, response format และ error codes

D. Function signature, return type, unit tests, documentation link และ changelog

⏸ หยุดคิดสัก 30 วินาที แล้วค่อยไปดูเฉลยหน้าถัดไป

✓ เฉลย Exam Sim — ข้อ 5/5

คำตอบที่ถูก: Option B

5 องค์ประกอบนี้ — purpose, inputs (พร้อม format/constraints), example queries, edge cases/limitations และ explicit boundaries — ให้ context เพียงพอที่โมเดลจะแยกความต่างของ tool และเลือกได้อย่างน่าเชื่อถือ

ทำไมข้ออื่นผิด

A — เป็น package metadata ไม่ใช่องค์ประกอบ description ที่ช่วย LLM เลือก tool

C — เป็น API specification fields แม้จะทับซ้อนบ้าง แต่ขาด example queries และ boundaries ที่สำคัญต่อการเลือก

D — เป็น software documentation fields ไม่ใช่องค์ประกอบ description ที่ช่วยให้ LLM เลือกระหว่าง tool